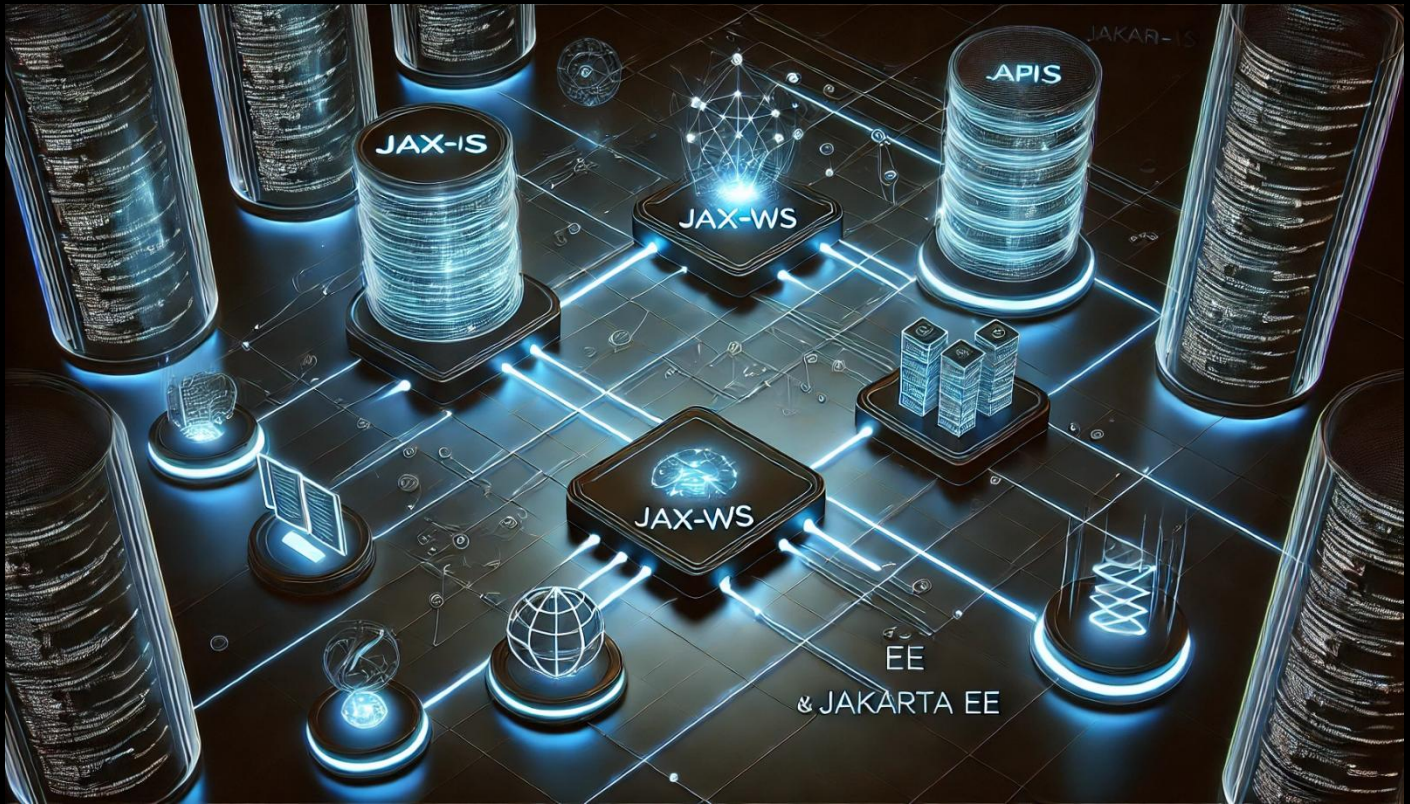


Web Services con JAX-WS y Jakarta EE



🧩 Guía: Creación de un Web Service con JAX-WS usando EJB

🎯 Objetivo del ejercicio

En esta lección vamos a exponer un servicio web usando Java EE con EJBs (Enterprise Java Beans) y JAX-WS (Java API for XML Web Services). Este servicio consistirá de lo siguiente: una función que suma dos números enteros.

🔧 Paso 1: Crear el proyecto Maven

1. En NetBeans, crea un nuevo proyecto:
 - File > New Project
 - Elige: **Java with Maven > Java Application**
 - Nombre: `sumaws`
 - Group Id: `sga`
 - Ubicación: `C:\Cursos\Java\JakartaEE`
 - Quita el nombre del paquete (déjalo vacío)
 - Clic en **Finalizar**



Paso 2: Modificar `pom.xml` para agregar dependencias

Abre el archivo:

`sumaws/pom.xml`

Justo después de la etiqueta `<properties>`, agrega las siguientes dependencias:

```

<dependency>
  <groupId>jakarta.platform</groupId>
  <artifactId>jakarta.jakartaee-api</artifactId>
  <version>${jakartaee}</version>
  <scope>provided</scope>
</dependency>
<dependency>
  <groupId>jakarta.jws</groupId>
  <artifactId>jakarta.jws-api</artifactId>
  <version>3.0.0</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>2.0.17</version>
  <type>jar</type>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>2.0.17</version>
  <scope>runtime</scope>
</dependency>

```

Estas dependencias permiten trabajar con Java EE y el API de servicios web. El `scope` es `provided` porque lo proporcionará el servidor (GlassFish). Además hemos agregado las dependencias de `slf4j` para el manejo de log de nuestra aplicación.



Archivo `pom.xml` completo:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>sga</groupId>

```

```

<artifactId>sumaws</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
  <exec.mainClass>mx.com.gm.sga.sumaws.Sumaws</exec.mainClass>
  <jakartaee>11.0.0-M1</jakartaee>
</properties>
<dependencies>
  <dependency>
    <groupId>jakarta.platform</groupId>
    <artifactId>jakarta.jakartaee-api</artifactId>
    <version>${jakartaee}</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>jakarta.jws</groupId>
    <artifactId>jakarta.jws-api</artifactId>
    <version>3.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.17</version>
    <type>jar</type>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-simple</artifactId>
    <version>2.0.17</version>
    <scope>runtime</scope>
  </dependency>
</dependencies>
</project>

```



Paso 3: Descargar dependencias

Haz clic derecho sobre el proyecto y selecciona:

Build with Dependencies (o Clean and Build)

Con esto NetBeans descargará los .jar necesarios y podrás ver el paquete `jakarta.jws-api-3.0.0.jar` disponible de las Dependencias del proyecto.

Paso 4: Crear la interfaz del Web Service

Ubicación del archivo:

`sumaws/src/main/java/beans/ServicioSumarWS.java`

Crea la interfaz con el siguiente contenido:

```
package beans;

import jakarta.jws.WebMethod;
import jakarta.jws.WebService;

@WebService
public interface ServicioSumarWS {
    @WebMethod
    int sumar(int a, int b);
}
```

 Explicación:

- `@WebService`: indica que esta interfaz se va a exponer como servicio web.
- `@WebMethod`: cada método con esta anotación será visible desde el servicio web.

Paso 5: Crear la clase que implementa el servicio

Ubicación del archivo:

`sumaws/src/main/java/beans/ServicioSumarImpl.java`

Contenido del archivo:

```
package beans;

import jakarta.ejb.Stateless;
import jakarta.jws.WebService;

@Stateless
@WebService(endpointInterface = "beans.ServicioSumarWS")
public class ServicioSumarImpl implements ServicioSumarWS{
```

```
@Override
public int sumar(int a, int b) {
    return a + b;
}

}
```

Explicación:

- `@Stateless`: indica que este EJB no guarda estado del cliente.
- `endpointInterface`: vincula esta clase con la interfaz `ServicioSumarWS`.
- El método `sumar` implementa la lógica real: sumar dos enteros.

Paso 6: Compilar y desplegar en GlassFish

1. Haz clic derecho en el proyecto:
Clean and Build
2. Abre GlassFish:
 - Clic derecho > Start
3. Una vez iniciado:
 - Admin Console > Applications > Deploy
 - Selecciona el archivo `.jar` generado (ruta):
 - `C:\Cursos\Java\JakartaEE\sumaws\target\sumaws-1.0.jar`
 - Nombre de despliegue: `sumaws`
 - Clic en **OK**

Paso 7: Probar el Web Service

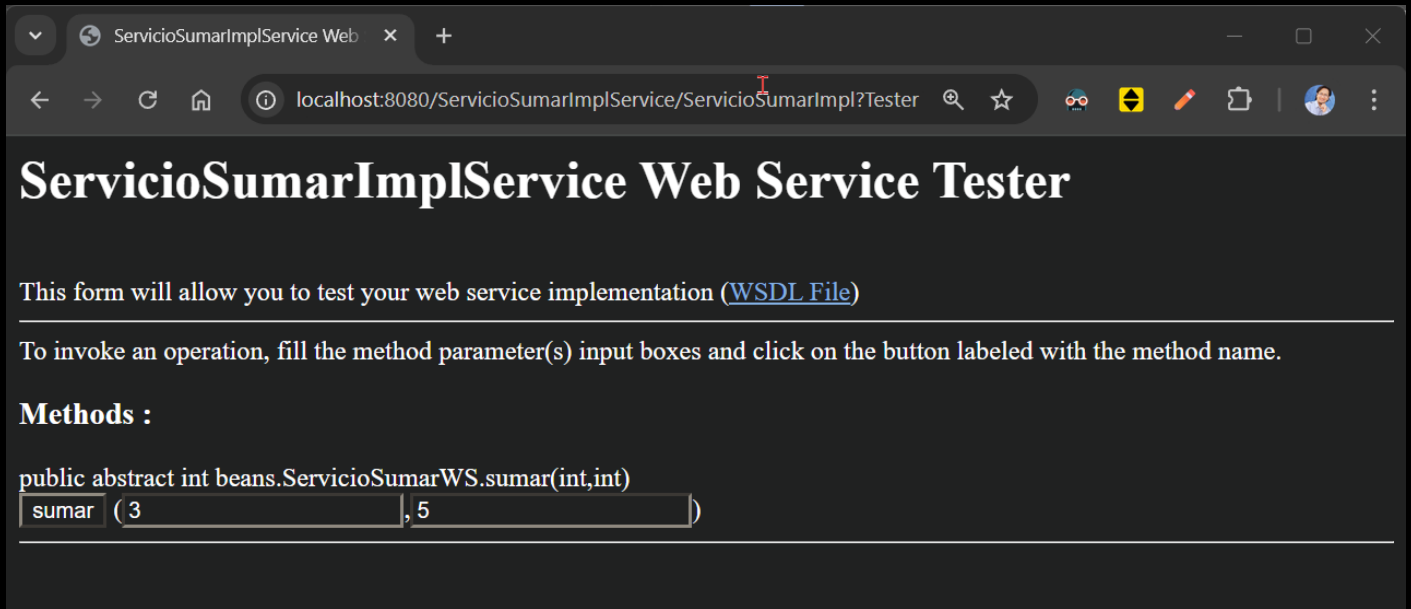
Después del despliegue, en la consola de administración:

1. Ve a:
`Applications > sumaws > ServicioSumarImpl > View Endpoint`
2. Verás una URL como esta:

<http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl?Tester>

Al hacer clic, verás un formulario para probar el método `sumar`.

Prueba con: `a = 3` y `b = 5` y presiona el botón de "sumar"



ServicioSumarImplService Web Service Tester

This form will allow you to test your web service implementation ([WSDL File](#))

To invoke an operation, fill the method parameter(s) input boxes and click on the button labeled with the method name.

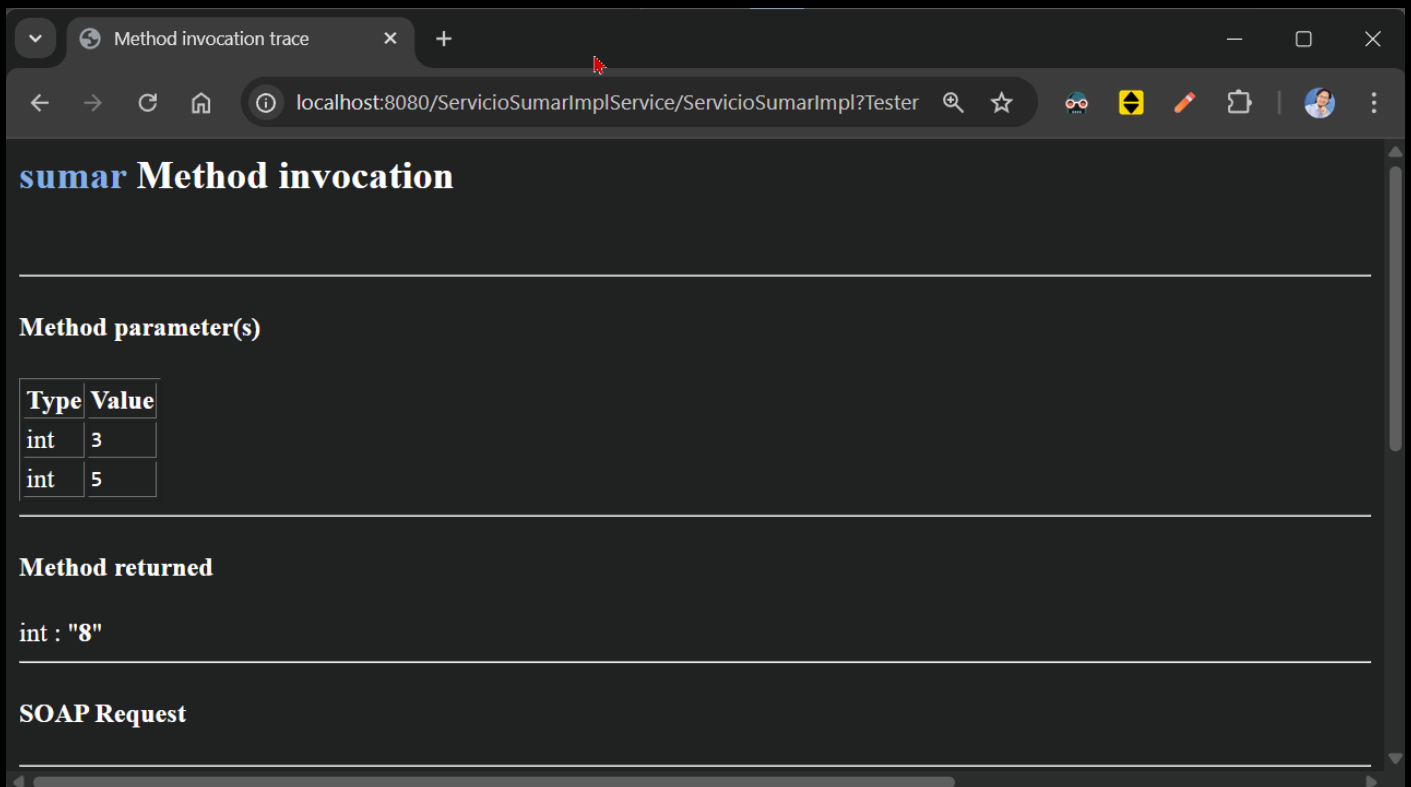
Methods :

```
public abstract int beans.ServicioSumarWS.sumar(int,int)
```

sumar (3 , 5)

Resultado esperado:

```
<sumarResponse>
  <return>8</return>
</sumarResponse>
```



Method invocation trace

localhost:8080/ServicioSumarImplService/ServicioSumarImpl?Tester

sumar Method invocation

Method parameter(s)

Type	Value
int	3
int	5

Method returned

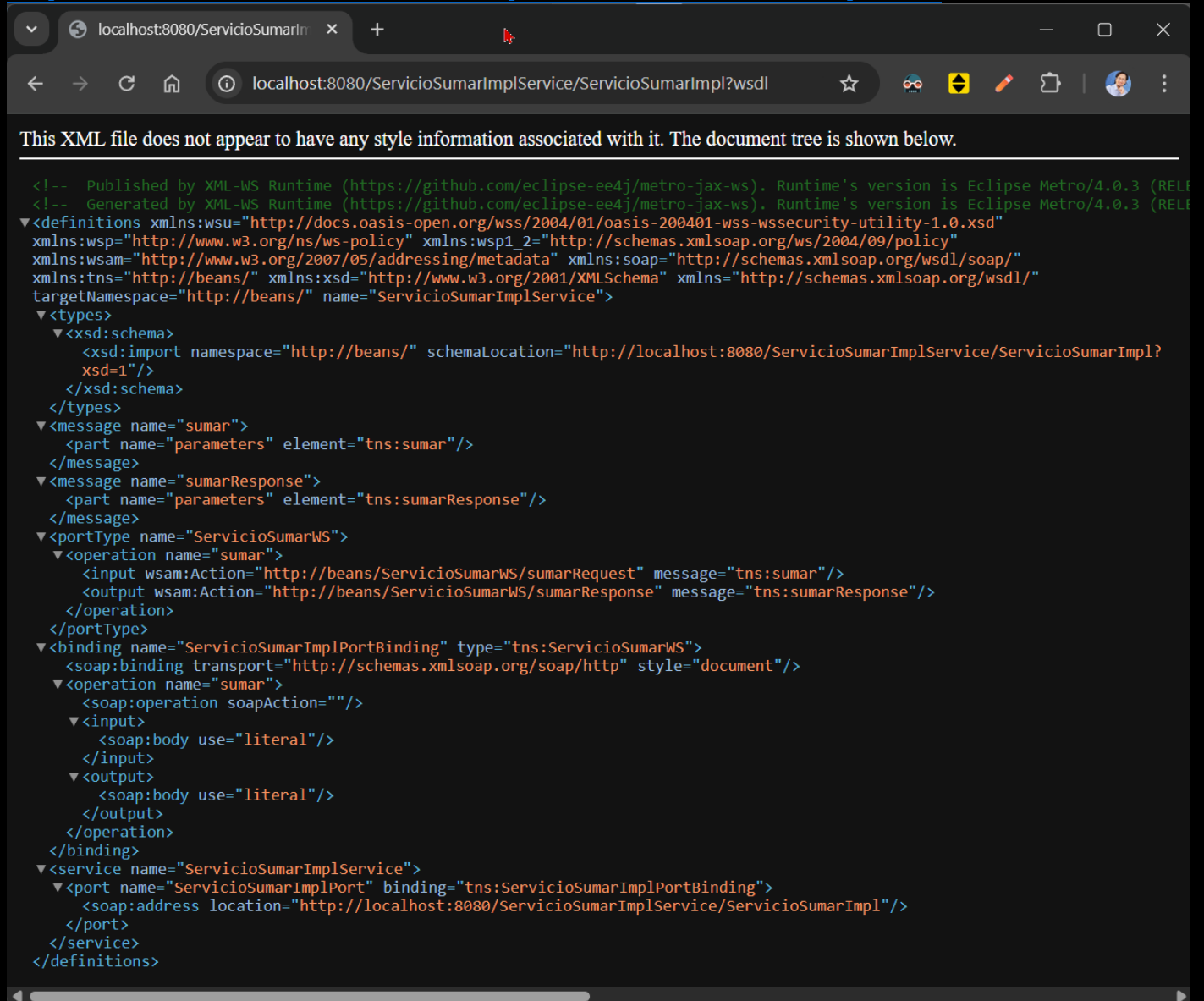
int : "8"

SOAP Request

✅ También puedes ver el **WSDL** (archivo de descripción del servicio web) en:

<https://www.globalmentoring.com.mx>

<http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl?wsdl>



```

<!-- Published by XML-WS Runtime (https://github.com/eclipse-ee4j/metro-jax-ws). Runtime's version is Eclipse Metro/4.0.3 (RELEASE) -->
<!-- Generated by XML-WS Runtime (https://github.com/eclipse-ee4j/metro-jax-ws). Runtime's version is Eclipse Metro/4.0.3 (RELEASE) -->
<?xml version="1.0" encoding="UTF-8"?>
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
  xmlns:wsp="http://www.w3.org/ns/ws-policy" xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy"
  xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://beans/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/"
  targetNamespace="http://beans/" name="ServicioSumarImplService">
  <types>
    <xsd:schema>
      <xsd:import namespace="http://beans/" schemaLocation="http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl?wsdl"
        xsd="1"/>
    </xsd:schema>
  </types>
  <message name="sumar">
    <part name="parameters" element="tns:sumar"/>
  </message>
  <message name="sumarResponse">
    <part name="parameters" element="tns:sumarResponse"/>
  </message>
  <portType name="ServicioSumarWS">
    <operation name="sumar">
      <input wsam:Action="http://beans/ServicioSumarWS/sumarRequest" message="tns:sumar"/>
      <output wsam:Action="http://beans/ServicioSumarWS/sumarResponse" message="tns:sumarResponse"/>
    </operation>
  </portType>
  <binding name="ServicioSumarImplPortBinding" type="tns:ServicioSumarWS">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
    <operation name="sumar">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
  <service name="ServicioSumarImplService">
    <port name="ServicioSumarImplPort" binding="tns:ServicioSumarImplPortBinding">
      <soap:address location="http://localhost:8080/ServicioSumarImplService/ServicioSumarImpl"/>
    </port>
  </service>
</definitions>

```

✿ ¿Qué es un archivo WSDL?

El archivo **WSDL** (Web Services Description Language) es un archivo **XML** que describe **completamente** un servicio web. Actúa como el **contrato** que le dice a cualquier cliente **cómo comunicarse** con el servicio web.

El WSDL es la carta de presentación técnica de un Web Service.

Dice qué métodos hay, qué parámetros esperan, cómo se llaman y dónde se encuentran.

🤖 ¿Para qué sirve?

1. 📄 Los clientes lo usan para generar el código que necesita para consumir el servicio.
 2. 🔒 Garantiza que el cliente y el servidor se entiendan, aunque estén escritos en distintos lenguajes (Java, .NET, PHP, etc.).
 3. 🤖 Facilita herramientas automáticas como `wsimport` (Java), que generan clases cliente directamente desde el WSDL.
-

Conclusión

En esta lección aprendiste a:

- Crear un Web Service con JAX-WS y EJB (`@WebService`, `@Stateless`)
 - Configurar dependencias necesarias
 - Exponer y consumir servicios SOAP
 - Usar GlassFish como servidor de despliegue
-

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)